DB201 Database Systems — Course Notes

Units 1–5: Relational Model, Normalization, SQL Joins, Transactions and Indexing

Unit 1: The Relational Model

A **relational database** stores data in **relations**, usually called tables. Each table has a fixed set of named **attributes** (columns), and each row of the table is called a **tuple**. The set of allowed values for an attribute is its **domain**; for example, the domain of an `age` attribute might be the non-negative integers.

The **schema** of a relation is its name together with its list of attributes, for example `Student(student_id, name, email, programme)`. The **instance** of a relation is the set of tuples it contains at a particular moment. Schemas change rarely; instances change every time data is inserted, updated or deleted.

Keys

A **superkey** is any set of attributes whose values uniquely identify a tuple in the relation. A **candidate key** is a minimal superkey: if any attribute is removed from it, it no longer uniquely identifies tuples. A relation may have several candidate keys. The database designer chooses one of them as the **primary key**, which is used to identify rows and must never be NULL.

A **foreign key** is an attribute (or set of attributes) in one relation that refers to the primary key of another relation. For example, `Enrollment.student_id` is a foreign key referencing `Student.student_id`. The rule that every foreign key value must match an existing primary key value (or be NULL) is called **referential integrity**. When a referenced row is deleted, the database can reject the delete (RESTRICT), delete the referencing rows as well (CASCADE), or set the foreign key to NULL (SET NULL).

A **surrogate key** is an artificial identifier, such as an auto-incrementing integer or a UUID, that has no business meaning. A **natural key** is built from real-world data, such as an email address or a course code. Surrogate keys are stable even when real-world data changes, which is why many systems prefer them for primary keys.

Unit 2: Functional Dependencies and Normalization

A **functional dependency** $X \rightarrow Y$ holds in a relation when any two tuples that agree on the

attributes X must also agree on the attributes Y. For example, in `Student(student_id, name, email)`, the dependency `student_id → name, email` holds because a student ID determines exactly one name and email.

Normalization is the process of decomposing relations to remove redundancy and the **update anomalies** it causes:

- An **insertion anomaly** occurs when some data cannot be recorded without also recording unrelated data, for example being unable to store a new course until a student enrols in it.
- A **deletion anomaly** occurs when deleting one fact unintentionally deletes another, for example removing the last student in a course also removes the only record of the course's instructor.
- An **update anomaly** occurs when the same fact is stored in many rows, so changing it requires updating every copy; missing one leaves the data inconsistent.

Normal forms

First Normal Form (1NF) requires every attribute to contain only atomic (indivisible) values and every row to be unique. A column that stores a comma-separated list of phone numbers violates 1NF; the fix is to move the phone numbers into a separate table with one number per row.

Second Normal Form (2NF) requires the relation to be in 1NF and every non-key attribute to depend on the *whole* of every candidate key, not just part of it. 2NF only matters when a key is composite. In `Enrollment(student_id, course_id, student_name, grade)`, `student_name` depends only on `student_id`, which is a **partial dependency**; moving `student_name` into the `Student` table brings the relation into 2NF.

Third Normal Form (3NF) requires the relation to be in 2NF and to have no **transitive dependencies**, meaning no non-key attribute depends on another non-key attribute. In `Course(course_id, department_id, department_name)`, `department_name` depends on `department_id`, which depends on `course_id`. Moving the department attributes into a separate `Department` table removes the transitive dependency.

Boyce–Codd Normal Form (BCNF) is a slightly stronger version of 3NF: for every non-trivial functional dependency X → Y, X must be a superkey. Every BCNF relation is in 3NF, but a 3NF relation is not always in BCNF. Decomposing into BCNF always gives a **lossless join**, but it may not preserve every functional dependency, which is why designers sometimes stop at 3NF.

Denormalization is the deliberate reintroduction of redundancy to make frequent read queries faster, at the cost of more complex updates. It should only be done after measuring a real performance problem.

Unit 3: SQL Joins

A **join** combines rows from two tables based on a related column. The examples below use `Student(student_id, name)` and `Enrollment(student_id, course_id)`.

- An **INNER JOIN** returns only the rows that have matching values in both tables. Students without any enrollment do not appear in the result.
- A **LEFT OUTER JOIN** returns every row from the left table and the matching rows from the right table. When there is no match, the columns from the right table are filled with NULL. `Student LEFT JOIN Enrollment` therefore lists every student, including those who have not enrolled in any course.
- A **RIGHT OUTER JOIN** is the mirror image: every row from the right table is kept.
- A **FULL OUTER JOIN** keeps every row from both tables, filling in NULL wherever there is no match on the other side.
- A **CROSS JOIN** returns the Cartesian product: every row of the first table paired with every row of the second. Two tables of 100 and 50 rows produce 5,000 rows.
- A **SELF JOIN** joins a table to itself using two aliases, for example to pair each employee with their manager when both are stored in the same `Employee` table.

A common mistake is filtering an outer-joined table in the `WHERE` clause. A condition such as `WHERE Enrollment.course_id = 'DB201'` removes the NULL rows produced by a LEFT JOIN and silently turns it back into an inner join. To keep unmatched rows, put the condition in the `ON` clause instead.

To find rows that have **no** match, use a LEFT JOIN and keep the rows where the right-hand key IS NULL. For example, students who are not enrolled in any course are those where `Enrollment.student_id IS NULL` after `Student LEFT JOIN Enrollment`.

Unit 4: Transactions and Concurrency Control

A **transaction** is a sequence of database operations that is treated as a single logical unit of work, such as transferring money between two accounts. A transaction ends either with **COMMIT**, which makes its changes permanent, or **ROLLBACK**, which undoes all of its changes.

ACID properties

- **Atomicity**: either all of the transaction's operations take effect or none of them do. If the system crashes halfway through a transfer, the debit is undone so money is never lost.
- **Consistency**: a transaction moves the database from one valid state to another, preserving all declared constraints such as primary keys, foreign keys and CHECK constraints.
- **Isolation**: concurrently running transactions do not see each other's intermediate, uncommitted changes. The result is as if the transactions had run one after another in some order.
- **Durability**: once a transaction has committed, its changes survive crashes and power failures. Databases achieve this with a **write-ahead log (WAL)**: changes are written to a log on disk before the data pages are modified, so committed work can be replayed after a crash.

Concurrency anomalies

- A **dirty read** happens when a transaction reads data written by another transaction that

has not yet committed. If that other transaction rolls back, the first transaction has used data that never officially existed.
- A **non-repeatable read** happens when a transaction reads the same row twice and gets different values because another transaction committed an update in between.
- A **phantom read** happens when a transaction runs the same query twice and the second run returns additional or missing rows because another transaction inserted or deleted matching rows.

Isolation levels

The SQL standard defines four isolation levels, from weakest to strongest:

1. **READ UNCOMMITTED** allows dirty reads, non-repeatable reads and phantoms.
2. **READ COMMITTED** prevents dirty reads, but non-repeatable reads and phantoms are still possible. This is the default level in PostgreSQL.
3. **REPEATABLE READ** prevents dirty reads and non-repeatable reads; the standard still allows phantoms.
4. **SERIALIZABLE** prevents all three anomalies; the outcome is equivalent to running the transactions one at a time.

Stronger isolation gives safer results but reduces concurrency, because transactions must wait for each other or be aborted and retried more often.

Locking

In **two-phase locking (2PL)**, each transaction has a *growing phase*, in which it may acquire locks but not release them, followed by a *shrinking phase*, in which it may release locks but not acquire new ones. 2PL guarantees serializable schedules. **Shared (S) locks** allow many transactions to read the same item; an **exclusive (X) lock** is needed to write it and blocks all other locks on that item.

A **deadlock** occurs when two or more transactions each wait for a lock held by another, so none can proceed. Databases detect deadlocks by looking for cycles in a **wait-for graph** and resolve them by aborting one of the transactions, which the application should then retry.

Unit 5: Indexing

An **index** is an auxiliary data structure that lets the database find rows matching a condition without scanning the whole table. Indexes speed up reads but slow down writes, because every INSERT, UPDATE and DELETE must also update each index on the table, and they use additional disk space.

B+ tree indexes

The **B+ tree** is the default index structure in most relational databases. It is a balanced tree: every leaf is at the same depth, so every lookup costs the same number of page reads, typically three or

four even for millions of rows. Internal nodes store only keys and child pointers, while all row pointers are stored in the leaves. The leaves are linked together in key order, which makes **range queries** such as `WHERE grade BETWEEN 60 AND 80` and `ORDER BY` very efficient.

Hash indexes

A **hash index** applies a hash function to the key to find the bucket that holds matching rows. Hash indexes support only **equality** lookups (`WHERE email = '...'`) and cannot answer range queries or help with sorting, because hashing destroys key order.

Clustered, composite and covering indexes

A **clustered index** determines the physical order of rows in the table, so a table can have at most one. A **non-clustered (secondary) index** is stored separately and points to the rows.

A **composite index** is built on several columns, such as `(last_name, first_name)`. It follows the **leftmost-prefix rule**: the index can be used for conditions on `last_name`, or on `last_name` and `first_name` together, but not on `first_name` alone.

A **covering index** contains every column a query needs, so the database can answer the query from the index alone (an index-only scan) without reading the table.

When to add an index

Good candidates for indexing are columns used frequently in `WHERE` clauses, join conditions and `ORDER BY`, especially when they are **selective** (they have many distinct values). Columns with very few distinct values, such as a boolean flag, rarely benefit from an index, and tables that are written far more often than they are read may become slower overall when indexes are added.